

Sentinel Operating Guide

Documentacao operacional, guia do usuario e diagnostico de fluxo do ecossistema Sentinel.

Estado observado em: 16/06/2026.

1. Objetivo deste documento

Este documento existe para responder tres perguntas:

1. O que e o Sentinel hoje?
2. Como usar o Sentinel Core no trabalho real?
3. Por que o fluxo atual pode nao estar agradando e onde ajustar primeiro?

Ele nao e uma promessa comercial, nem um roadmap idealizado. Ele descreve o estado pratico do sistema hoje, separando o que ja funciona como operacao real do que ainda esta em transicao.

2. Visao geral do ecossistema

A Sentinel hoje e formada por cinco superficies principais:

Superficie	URL / caminho	Papel
Site publico	https://sentinelqa.tech	Frente institucional da Sentinel Tech - QA.
Sentinel Core	https://sentinelqa.tech/core/dashboard	Plataforma interna de operacao QA.
Sentinel Learning	https://learning.sentinelqa.tech/	Area de estudos e trilhas.
Sentinel Extension	https://sentinelqa.tech/extension/	Apoio para captura/importacao de informacoes de QA.
API	apps/api	Backend estrutural, ainda em consolidacao como fonte real.

Rotas legadas e atalhos:

URL	Comportamento esperado
https://sentinelqa.tech/core	Redireciona para o Core.
https://app.sentinelqa.tech	Redireciona para o Core.
https://sentinelqa.tech/learning	Redireciona para learning.sentinelqa.tech .
https://sentinelqa.tech/learning/	Redireciona para learning.sentinelqa.tech .

3. Estado atual dos produtos

3.1 Site publico

O site publico e a porta de entrada institucional.

Responsabilidades:

- Apresentar a marca Sentinel Tech - QA.
- Comunicar servicos.
- Direcionar contato.
- Servir como frente para Core, Learning e Extension.

Stack:

- HTML.
- CSS.
- JavaScript simples.
- Vercel, projeto [sentinel-website](#).

Observacao:

O site publico nao e o lugar onde a operacao acontece. Ele deve ser simples, rapido e confiavel.

3.2 Sentinel Core

O Core e a plataforma interna de operacao.

Responsabilidades atuais:

- Dashboard operacional.
- Daily.
- QA Importer / Tasks.
- Board.
- Bugs.
- Sprints.
- Projects.
- Clients.
- Team.
- Calendar.
- Reports.
- Notifications.
- Settings.

Stack:

- Next.js.
- React.
- TypeScript.
- Tailwind.
- Zustand.

- TanStack Query.
- Supabase Auth.
- API propria em transicao.

Observacao importante:

O Core esta em uma fase hibrida. Algumas areas ja usam hooks de API/React Query; outras ainda usam Zustand/localStorage. Isso afeta a sensacao de fluxo, persistencia e confiabilidade.

3.3 Sentinel Learning

O Learning atual vive fora do monorepo principal, em um projeto Vercel separado:

```
D:\DEV\01_COMPANIES\SentinelQAtech\sentinel-learning
```

URL:

```
https://learning.sentinelqa.tech/
```

Responsabilidade:

- Conteudos de estudo.
- Trilhas.
- Cursos.
- Capacitacao interna ou futura oferta educacional.

Observacao:

Learning nao faz parte do fluxo diario do Core. Ele deve ser tratado como area de apoio, nao como etapa obrigatoria da operacao QA.

3.4 Sentinel Extension

A Extension esta versionada em:

```
tools/core-extension
```

Responsabilidade prevista:

- Apoiar captura de dados externos.
- Reduzir trabalho manual.
- Enviar informacoes para o fluxo do Core/QA Importer.

Estado atual:

- Existe como parte do ecossistema.

- Ainda precisa estar claramente conectada a um fluxo operacional fechado.

3.5 API

A API existe em:

```
apps/api
```

Responsabilidade prevista:

- Ser fonte real de dados.
- Consolidar usuarios, clientes, projetos, bugs, tasks, sprints, reports e notificacoes.

Estado atual:

- Estrutural.
- Parcialmente integrada.
- Ainda nao e a fonte unica de verdade do Core.

4. Guia do usuario do Sentinel Core

4.1 Entrada no sistema

Fluxo:

1. Acesse <https://sentinelqa.tech>.
2. Clique em [Core](#).
3. O sistema abre <https://sentinelqa.tech/core/login>.
4. Faça login.
5. Voce deve cair no Dashboard.

Estado atual:

- O login usa Supabase Auth.
- O dashboard protegido depende da sessao Supabase.
- O refresh da pagina deve manter a sessao quando o usuario esta logado.

4.2 Dashboard

Papel esperado:

O Dashboard deve ser o cockpit do dia.

Ele mostra:

- Saudacao do usuario.
- Filtros de visao: Tudo, Daily, QA, Riscos, Sprint.
- Widgets de Daily.
- Widgets de QA.

- Widgets de sprint.
- Projetos ativos.
- Bugs criticos.
- Calendario.
- Atalhos.
- Layout personalizavel.

Uso recomendado hoje:

1. Abrir o Dashboard no inicio do dia.
2. Filtrar por **Daily** para ver plano do dia.
3. Filtrar por **QA** para ver itens de teste/risco.
4. Filtrar por **Riscos** quando precisar priorizar.

Ponto de atencao:

O Dashboard pode comunicar "dados reais", mas parte dos dados ainda vem de stores locais. Portanto, ele deve ser lido como cockpit operacional em consolidacao, nao como relatorio final confiavel.

4.3 Daily

Papel esperado:

Daily deve organizar o trabalho de hoje.

O que ela deve responder:

- O que eu preciso fazer hoje?
- Quais reunioes existem?
- Quais itens estao em andamento?
- O que esta bloqueado?
- O que foi concluido?

Uso recomendado hoje:

1. Abrir **Daily**.
2. Conferir data selecionada.
3. Adicionar tarefas manuais quando necessario.
4. Adicionar reunioes.
5. Enviar itens do QA Importer para a Daily quando fizer sentido.
6. Marcar progresso durante o dia.

Ponto de atencao:

A Daily e um dos pontos mais importantes do Core, mas o fluxo dela ainda depende muito da relacao com QA Importer. Se essa relacao nao estiver clara, o usuario sente que a Daily e mais uma tela, nao o centro da rotina.

4.4 Tasks / QA Importer

Estado real hoje:

O item de menu **Tasks** abre o mesmo client do QA Importer.

Isso significa que, na pratica, **Tasks** nao e uma lista generica de tarefas. Ele esta muito mais proximo de uma caixa de entrada de trabalho QA.

Papel esperado:

- Importar cards.
- Normalizar itens de QA.
- Classificar prioridade.
- Enviar itens para Daily.
- Acompanhar status diario.
- Registrar resolucao e evidencia.
- Arquivar sessoes.

Uso recomendado hoje:

1. Abrir **Tasks**.
2. Importar itens manualmente, por CSV ou pela Extension.
3. Revisar cliente, projeto, prioridade, sprint e categoria.
4. Enviar itens relevantes para a Daily.
5. Durante a execucao, atualizar estado do item.
6. Ao concluir validacao, registrar resolucao:
 - PASS.
 - FAIL.
 - PARTIAL.
 - BLOCKED.
7. Usar o relatorio gerado como base de evidencia.

Ponto de atencao:

Este modulo provavelmente deveria ser nomeado com mais clareza. Hoje **Tasks** pode criar expectativa errada. Possiveis nomes melhores:

- **QA Inbox**.
- **QA Work**.
- **QA Importer**.
- **Testing Queue**.
- **Work Intake**.

4.5 Board / Kanban

Papel esperado:

O Board deve mostrar o estado de execucao dos itens QA.

Colunas atuais:

- Ready for QA.
- In Testing.
- Bug Validation.

- Regression.
- Review.
- Blocked.
- Done.

Uso recomendado hoje:

1. Usar o Board para visualizar fluxo.
2. Mover itens conforme estado real.
3. Manter **Blocked** reservado para bloqueios reais.
4. Manter **Done** apenas para itens validados.

Ponto de atencao:

O Board esta alinhado com categorias do QA Importer. Ele funciona melhor quando QA Importer e Daily sao usados como entrada do fluxo.

4.6 Bugs

Papel esperado:

Bugs deve registrar risco, falha e evidencia.

Estado real hoje:

- O store de Bugs ainda existe como legado.
- Parte do modulo ja aponta para hooks de API/React Query.
- Ha sincronizacao de itens do QA Importer para Bugs.

Uso recomendado hoje:

1. Usar Bugs para visualizar problemas importados ou derivados de QA.
2. Registrar bugs manuais quando necessario.
3. Tratar bugs como evidencia de risco, nao como duplicacao de todos os cards.

Ponto de atencao:

Se todo item QA vira bug, o modulo perde significado. Um ajuste de produto pode ser separar:

- Trabalho de QA.
- Bug real.
- Risco.
- Validacao pendente.

4.7 Sprints

Papel esperado:

Sprints deve agrupar trabalho por ciclo.

Uso recomendado hoje:

1. Usar Sprint para contexto de planejamento.

2. Relacionar itens importados ao sprint correto.
3. Observar progresso e burndown quando houver dados suficientes.

Ponto de atencao:

Sprints so fica util quando os itens do fluxo possuem sprint real preenchida.

4.8 Projects

Papel esperado:

Projects deve organizar trabalho por projeto.

Estado real hoje:

- Existe hook/API para projetos.
- Tambem existe store legado para sincronizacao vinda do QA Importer.

Uso recomendado hoje:

1. Criar/usar projetos reais.
2. Evitar usar cliente como substituto automatico de projeto.
3. Preencher projeto no QA Importer sempre que o item pertencer a um projeto real.

Ponto de atencao:

Se cliente, projeto e sprint nao estiverem bem definidos, o Core tende a virar uma lista grande de itens soltos.

4.9 Clients

Papel esperado:

Clients deve representar empresas/clientes atendidos.

Uso recomendado hoje:

1. Manter cadastro de clientes reais.
2. Associar projetos e itens QA aos clientes.
3. Usar cliente como dimensao de filtro e relatorio.

Ponto de atencao:

Clientes sao uma dimensao de negocio. Projetos sao uma dimensao de trabalho. Os dois nao deveriam ser confundidos.

4.10 Team

Papel esperado:

Team deve representar pessoas ativas e historico de participacao.

Uso recomendado hoje:

1. Cadastrar membros reais.

2. Usar funcoes e contexto para entender capacidade.
3. Evitar depender de dados ficticios para operacao.

Ponto de atencao:

Team fica mais valioso quando conectado a responsaveis reais em itens, bugs, projetos e dailies.

4.11 Calendar

Papel esperado:

Calendar deve organizar eventos, reunioes, deadlines e reviews.

Uso recomendado hoje:

1. Registrar reunioes importantes.
2. Registrar deadlines.
3. Usar como apoio da Daily.

Ponto de atencao:

Calendar nao deve ser um silo. Ele precisa aparecer na Daily e no Dashboard para ter utilidade diaria.

4.12 Reports

Papel esperado:

Reports deve fechar ciclos e comunicar resultado.

Uso recomendado hoje:

1. Revisar itens resolvidos.
2. Usar dados de QA Importer, Bugs e Daily.
3. Gerar base para comunicacao interna ou cliente.

Ponto de atencao:

Reports so se torna confiavel quando a entrada de dados tambem e confiavel.

4.13 Notifications

Papel esperado:

Notifications deve avisar eventos importantes.

Estado real hoje:

- Ha hook de notificacoes.
- Ha estrutura de socket.
- Ainda depende da consolidacao da API.

Uso recomendado hoje:

Usar como apoio, nao como fonte principal da rotina.

4.14 Settings e Profile

Papel esperado:

- Profile: dados do usuario.
- Settings: preferencias do sistema.
- Language Toggle: alternancia de idioma.

Ponto de atencao:

A traducao ainda deve ser observada com cuidado. O objetivo declarado e traduzir a experiencia inteira, nao apenas o menu.

5. Fluxo de trabalho real de hoje

Este e o fluxo mais proximo do que o Core suporta hoje.

```
Site publico
-> Core
  -> Dashboard
    -> QA Importer / Tasks
      -> Daily
        -> Board
          -> Bugs / Resolucao / Evidencias
            -> Reports
```

Passo a passo:

1. Entrar no Core.
2. Abrir Dashboard para entender o dia.
3. Abrir Tasks/QA Importer para alimentar o trabalho.
4. Importar ou criar itens QA.
5. Revisar prioridade, cliente, projeto, sprint e categoria.
6. Enviar itens relevantes para Daily.
7. Usar Daily como plano de execucao do dia.
8. Usar Board para visualizar estado macro.
9. Quando encontrar problema, registrar ou acompanhar em Bugs.
10. Ao finalizar, registrar resolucao e evidencia.
11. Usar Reports para fechamento.

O ponto central do fluxo hoje nao e "tarefa generica". O ponto central e:

```
Item QA importado ou criado -> priorizacao -> Daily -> execucao -> evidencia.
```

6. Fluxo ideal recomendado

Se a meta é trabalhar melhor no Core, o fluxo deveria ser simplificado para:

1. Inbox
2. Today
3. Execution
4. Evidence
5. Report

Traduzindo para o produto atual:

Etapas ideais	Módulo atual mais próximo	Função
Inbox	Tasks / QA Importer	Entrada e triagem de trabalho.
Today	Daily	Plano do dia.
Execution	Board	Estado de execução.
Evidence	QA resolution / Bugs	Resultado, falha, bloqueio e evidência.
Report	Reports	Fechamento e comunicação.

Recomendação:

O Core deveria deixar esse fluxo explícito na interface. Hoje ele existe no código e nos módulos, mas não está claro o bastante para guiar o usuário.

7. Onde o fluxo está confuso hoje

7.1 **Tasks** não parece **Tasks**

O menu diz **Tasks**, mas abre o QA Importer.

Impacto:

- O usuário espera uma lista de tarefas simples.
- O sistema mostra um fluxo de importação/classificação QA.
- A expectativa quebra logo no início.

Possível decisão:

Renomear o menu para **QA Inbox**, **QA Work** ou **QA Importer**.

7.2 Quick Create não parece persistir no fluxo real

O botão **Criar** abre um modal para criar tarefa, bug, projeto ou sprint.

Problema:

Pelo estado atual do componente, ele exibe feedback visual, mas não parece conectar claramente a stores/API reais.

Impacto:

- O usuario sente que criou algo, mas nao ve o item entrar no fluxo.
- Isso quebra confianca.

Possivel decisao:

Ou conectar Quick Create ao fluxo real, ou esconder ate estar pronto.

7.3 Dashboard promete mais do que a persistencia garante

O Dashboard fala em dados reais do workspace.

Problema:

O Core ainda mistura:

- Supabase Auth.
- API propria.
- React Query.
- Zustand.
- localStorage.
- dados derivados do QA Importer.

Impacto:

- O usuario nao sabe o que e real, salvo, local ou calculado.
- Limpar cache pode afetar partes do trabalho.

Possivel decisao:

Marcar internamente quais widgets sao confiaveis e quais ainda dependem de persistencia local.

7.4 Bugs e QA Work se sobrepõem

Hoje todos os itens QA podem aparecer como bugs.

Impacto:

- Bug deixa de significar "falha".
- O modulo vira outra visualizacao do mesmo trabalho.
- O usuario nao sabe se deve trabalhar em Tasks, Board ou Bugs.

Possivel decisao:

Separar categorias:

- QA Item.
- Bug real.
- Risk.
- Blocker.
- Evidence.

7.5 Daily e QA Importer precisam virar uma dupla clara

O melhor fluxo atual é enviar itens QA para a Daily.

Problema:

Isso precisa ficar evidente no produto. Se não ficar, o usuário tenta usar Daily como lista manual e Tasks como outra lista separada.

Possível decisão:

Fazer da Daily o centro do dia e do QA Importer a inbox.

7.6 Muitos módulos competem pela atenção

O Core tem muitos módulos:

- Dashboard.
- Daily.
- Tasks.
- Board.
- Bugs.
- Sprints.
- Projects.
- Clients.
- Team.
- Reports.
- Calendar.
- Notifications.
- Settings.

Impacto:

O usuário pode não saber onde começar.

Possível decisão:

Definir uma hierarquia:

1. Essenciais para trabalhar hoje.
2. Apoio.
3. Admin/configuração.
4. Futuro/experimental.

8. Modelo mental recomendado

O Sentinel Core deve ser pensado assim:

```
Dashboard = visão geral
QA Inbox = entrada de trabalho
```

Daily = compromisso do dia
Board = movimento do trabalho
Bugs = falhas e riscos reais
Reports = fechamento

Tudo que nao apoia esse fluxo deve ser tratado como secundario.

9. Rotina diaria sugerida

Inicio do dia

1. Abrir Dashboard.
2. Ver filtro **Daily**.
3. Ver filtro **QA**.
4. Abrir Tasks/QA Importer.
5. Revisar inbox.
6. Enviar para Daily o que sera feito hoje.

Durante o dia

1. Trabalhar pela Daily.
2. Atualizar status no Board quando o estado mudar.
3. Registrar bloqueios.
4. Registrar bugs reais quando encontrados.
5. Registrar evidencias em itens finalizados.

Fim do dia

1. Revisar Daily.
 2. Ver itens pendentes.
 3. Arquivar ou manter sessoes QA conforme necessidade.
 4. Gerar/consultar Reports.
 5. Definir o que fica para o proximo dia.
-

10. Rotina semanal sugerida

1. Revisar Projects.
 2. Revisar Sprints.
 3. Conferir bugs abertos.
 4. Conferir itens bloqueados.
 5. Limpar itens duplicados ou obsoletos.
 6. Gerar resumo semanal.
 7. Ajustar prioridades da proxima semana.
-

11. O que e fonte de verdade hoje

Area	Fonte atual mais provavel	Confiabilidade
Login	Supabase Auth	Alta
Dashboard	Mistura de stores e hooks	Media
Daily	Zustand/localStorage	Media
QA Importer	Zustand/localStorage por workspace	Media
Board	QA categories + store Kanban	Media
Bugs	API/React Query + store legado	Em transicao
Projects	API/React Query + store legado	Em transicao
Reports	Depende dos dados locais/API	Media/baixa
Notifications	API/socket em consolidacao	Em transicao
Learning	Projeto separado	Alta para conteudo estatico

Conclusao:

A principal fragilidade operacional ainda e persistencia/fonte de verdade.

12. Deciso es recomendadas

Decisao 1: Renomear **Tasks**

Recomendacao:

Renomear para **QA Inbox** ou **QA Work**.

Motivo:

O modulo atual nao e uma task list generica. Ele e a entrada de trabalho QA.

Decisao 2: Escolher o centro do dia

Recomendacao:

Daily deve ser o centro.

Motivo:

Dashboard e visao; QA Importer e entrada; Board e estado; Daily e compromisso.

Decisao 3: Reduzir modulos no fluxo principal

Fluxo principal sugerido:

```
Dashboard -> QA Inbox -> Daily -> Board -> Evidence/Reports
```

Todo o resto deve apoiar esse fluxo.

Decisao 4: Pausar ou corrigir Quick Create

Recomendacao:

Se Quick Create nao grava em dados reais, esconder ou conectar.

Motivo:

Botao que parece criar mas nao cria no fluxo real gera frustracao imediata.

Decisao 5: Consolidar persistencia

Recomendacao:

Priorizar:

1. QA Items.
2. Daily.
3. Bugs.
4. Projects.
5. Reports.

Motivo:

Esses sao os objetos que sustentam o trabalho real.

13. Backlog de melhoria de fluxo

Alta prioridade

- Renomear **Tasks** para algo alinhado ao uso real.
- Explicar visualmente a relacao **QA Inbox -> Daily**.
- Conectar Quick Create a dados reais ou remover da UI.
- Definir quais dados sobrevivem a limpeza de cache.
- Tornar Daily o centro do trabalho diario.

Media prioridade

- Separar QA Item de Bug real.
- Melhorar Reports com base em dados confiaveis.
- Conectar Calendar de forma mais forte a Daily.
- Definir fluxo de arquivamento de sessoes QA.
- Melhorar nomes e textos dos modulos.

Baixa prioridade

- Refinar Learning.
- Refinar Extension.
- Ajustar microinteracoes.

- Expandir internacionalizacao completa.

14. Perguntas para diagnostico com usuario real

Use estas perguntas quando sentir que o fluxo nao esta agradando:

1. Quando voce abre o Core, voce sabe qual tela deve usar primeiro?
2. Voce sente que **Tasks** representa o que aparece na tela?
3. Voce prefere planejar o dia pela Daily ou pelo Board?
4. O Dashboard te ajuda a decidir o que fazer agora?
5. Voce confia que o que criou foi salvo?
6. Voce entende quando um item deve virar Bug?
7. Voce sabe onde registrar evidencia?
8. Voce sabe como fechar o dia?
9. Voce sabe o que e cliente, projeto e sprint no sistema?
10. Qual tela voce abriria se tivesse apenas 5 minutos antes da daily real?

15. Guia rapido para trabalhar hoje

Se o objetivo e simplesmente trabalhar melhor hoje, use assim:

1. Entre no Core.
2. Abra **Tasks**.
3. Importe ou revise seus itens QA.
4. Envie para **Daily** apenas o que sera feito hoje.
5. Trabalhe pela **Daily**.
6. Use **Board** para movimentar estado.
7. Use **Bugs** apenas para falhas/riscos reais.
8. Registre resolucao/evidencia no item QA.
9. No fim do dia, revise pendencias e gere resumo.

Evite por enquanto:

- Usar Quick Create como fonte principal.
- Tratar Dashboard como relatorio final.
- Criar projetos sem associar cliente/sprint.
- Usar Bugs como lista de tudo.
- Dependder de dados locais como backup definitivo.

16. Proposta de novo fluxo de navegacao

Menu ideal para operacao real:

Home
Today
QA Inbox

Board
Bugs
Reports

Management
Clients
Projects
Sprints
Team

Support
Calendar
Learning
Settings

Comparacao com menu atual:

Atual	Problema	Sugestao
Dashboard	Ok, mas pode virar cockpit	Manter como Home
Daily	Bom centro do dia	Renomear para Today ou manter Daily
Tasks	Nome desalinhado	Renomear para QA Inbox
Board	Ok	Manter
Bugs	Sobreposicao com QA Item	Reservar para bugs reais
Sprints	Gestao/ciclo	Mover para Management
Projects	Gestao	Manter em Management
Clients	Gestao	Manter em Management
Team	Gestao	Manter em Management
Reports	Fechamento	Manter
Calendar	Apoio	Mover para Support ou integrar melhor com Daily
Learning	Fora do fluxo operacional	Manter como Support/external

17. Definicao operacional recomendada

Para o Sentinel Core ficar mais agradável, cada modulo precisa ter uma frase:

- Dashboard: "O que esta acontecendo agora?"
- Daily: "O que vou fazer hoje?"
- QA Inbox: "Que trabalho entrou para QA?"
- Board: "Em que estado esta cada item?"
- Bugs: "Quais falhas reais precisam de atencao?"
- Reports: "O que foi validado e como comunicar?"
- Projects: "Em quais projetos estamos trabalhando?"

- Clients: "Para quem estamos trabalhando?"
- Sprints: "Em qual ciclo estamos?"
- Team: "Quem esta envolvido?"
- Calendar: "Quando algo precisa acontecer?"

Se uma tela não responder uma frase simples, ela provavelmente está confusa.

18. Conclusão

O Sentinel já tem estrutura suficiente para operar, mas o fluxo atual ainda parece mais uma coleção de módulos do que uma rotina guiada.

O principal ajuste não é visual. O principal ajuste é de modelo mental:

```
Entrada -> Planejamento do dia -> Execução -> Evidência -> Fechamento
```

Hoje o sistema tem peças para isso, mas elas precisam ser nomeadas, conectadas e persistidas com mais clareza.

Recomendação final:

Começar pelo fluxo:

```
QA Inbox -> Daily -> Board -> Evidence -> Reports
```

Depois ajustar Dashboard para ser reflexo desse fluxo, não uma tela separada de indicadores.